Faculdade de Ciencias da Universidade de Lisboa

Break-it Phase Vulnerability Analysis

Secure Gallery Log Systems

Jose Sampaio, fc66191
Rodrigo Neto, fc59850
Gabriel Cambraia, 58671

Seguranca de Redes e Sistemas Distribuidos
2025/2026

Contents

1 Introduction

This document reports our vulnerability analysis for the Break-it phase of the SRSD Gallery Log project. We were assigned two independent implementations to examine: Group 1 and Group 21. For each implementation, we document how we built and ran the submitted programs, what parts of the security design we reviewed, which attacks we attempted, and whether we found a reproducible security vulnerability.

According to the project statement, valid Break-it findings include crashes, integrity violations, and confidentiality violations. We therefore distinguish security-relevant vulnerabilities from ordinary correctness bugs. For each successful attack, we provide a high-level summary, the root cause, reproduction steps, the expected behavior of a correct implementation, the observed vulnerable behavior, and references to any attack code or testcase files submitted alongside this report.

2 Evaluation Criteria Used

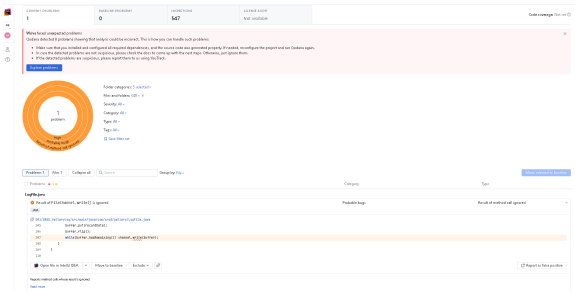
We evaluated each assigned implementation against the following security properties from the project specification:

- **Crash resistance:** malformed or adversarial log files should not cause abnormal termination; they should be rejected cleanly.
- **Integrity:** an attacker without the token should not be able to modify, delete, insert, reorder, or forge log entries so that `logread` accepts the resulting file.
- **Confidentiality:** an attacker without the token should not be able to infer private log contents or query results from the file representation alone.
- **Authentication:** commands using the wrong token should not be able to append to or read an existing log.
- **State consistency:** invalid gallery transitions should be rejected and should not leave the log in a partially modified state.

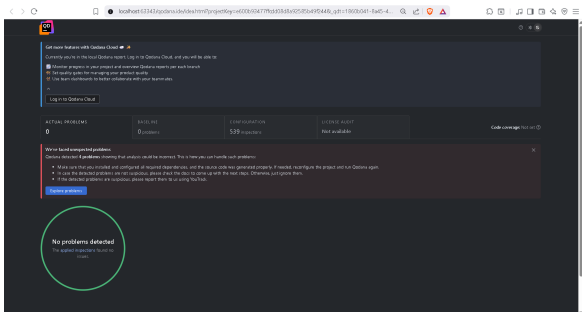
The attack templates below follow the testcase format requested in the project statement: create or obtain a valid log, perform the adversarial step without the token, and then show how the target implementation processes the resulting file incorrectly.

3 Static Analysis with Qodana

As an additional preliminary step, we ran JetBrains Qodana as a static-analysis tool over the assigned implementations. The goal was to identify automatically detectable security issues before performing manual code review and behavioral testing. Qodana did not report any security-relevant findings for either assigned project, so the vulnerabilities and behavioral deviations discussed later in this report were identified through manual analysis and targeted testing instead.



Group 1 Qodana report



Group 21 Qodana report

Figure 1: Static-analysis reports produced with Qodana. No security issues were reported by the tool.

4 Methodology: Shared Behavioral Test Suite

Before analyzing the assigned implementations, we developed one shared behavioral test suite and adapted only the wrapper commands for each target. Since the Group 1 and Group 21 demos exercise equivalent behavior, we do not include the full terminal transcripts in this report. Instead, we summarize what the demos test once in Table 1, and then report the relevant deviations for each group in their respective sections. The complete scripts and raw outputs are submitted as external artifacts.

Category	**Behavior tested**
Basic gallery flow	Employee and guest arrivals and departures at gallery level.
Room transitions	Entering rooms, leaving rooms, and preventing impossible room occupancy states.
Query behavior	Current state queries (`-S`), room-history queries (`-R`), and intersection queries (`-I`).
Invalid transitions	Duplicate arrivals, leaving without entering, leaving a room before entering it, timestamp regressions, invalid names, and conflicting flags.
Token handling	Rejection of operations attempted with an incorrect token.
Tamper handling	Rejection of logs modified directly on disk after valid creation.
Batch mode	Processing a batch file with valid lines and intentionally invalid lines while preserving safe log state.

Table 1: Shared behavioral coverage used for both assigned implementations

This condensed presentation keeps the report focused on vulnerability analysis rather than terminal output. For each group, we classify each behavioral deviation as either a security-relevant failure or a correctness/compatibility issue. In particular, incorrect exit codes are only treated as security issues when the unsafe action is actually accepted or the protected data is exposed.

5 Implementation G1: Group 1

5.1 Target Summary

- **Assigned implementation:** Group 1.
- **Language and build system:** Java 17 with Maven.
- **Programs analyzed:** `logappend` and `logread`.
- **Analysis status:** Successful vulnerabilities identified and reproduced.
- **Main result:** Behavioral testing and code review revealed multiple security failures: (1) the implementation reuses the same AES-GCM IV for all records in a log file, causing a catastrophic confidentiality failure; (2) the encrypted log file leaks information through predictable record sizes, allowing an attacker to infer the number of events and approximate name lengths; (3) the log filename argument is vulnerable to path traversal, allowing logs to be written to and read from arbitrary filesystem locations; and (4) the integrity mechanism does not detect rollback attacks—substituting an older version of the log file goes undetected, allowing an attacker to erase history and continue reading and writing as if the rolled-back state were valid.

5.2 How We Built and Ran the Implementation

```
cd SRSD_GalleryLog
mvn package

./logappend -T 1 -K segredo -A -E Alice galeria.log
./logappend -T 2 -K segredo -A -E Alice -R 5 galeria.log

./logread -K segredo -S galeria.log
```

Observed result: The implementation compiled successfully using Maven and generated two executable wrapper scripts, `logappend` and `logread`, which invoke the packaged JAR files. Basic functionality tests succeeded, including append operations, room transitions, history queries, integrity verification, and batch mode execution.

5.3 Behavioral Test Results

Table 2 summarizes the Group 1 execution of the shared test suite described in Section 4. As with Group 21, the full transcript is omitted from the report and submitted separately as an artifact.

Test area	Result	Relevant observation
Basic gallery flow	Pass	Normal append and state-query behavior worked as expected.
Room transitions	Pass	Room entry and exit transitions were enforced correctly.
State and intersection queries	Pass	`-S` and `-I` matched the expected scenarios.
Room history query	Minor issue	Revisited rooms were printed again, e.g., `3,7,3` instead of `3,7`.
Invalid actions	Pass	Invalid transitions were rejected without corrupting the log.
Wrong token and tampering	Pass	Wrong-token and tampered-log tests were rejected safely.
Batch mode	Issue	Batch handling deviated from the expected behavior in the tested invalid-line scenario.

Table 2: Condensed behavioral results for Group 1

5.4 Behavioral Findings

5.4.1 Room History Duplication

Group 1 shows the same `-R` behavior as Group 21: repeated visits to the same room are printed multiple times. For example, a visit sequence through rooms 3, 7, and 3 is reported as `3,7,3`, while the expected output is `3,7` if room history is interpreted as the list of distinct rooms visited in first-visit order. This is a correctness issue and not, by itself, a confidentiality or integrity break.

5.4.2 Batch Mode Deviation

The shared demo also exposed a batch-mode issue. The expected behavior is that valid batch lines are processed safely and invalid lines are rejected according to the project specification, without leaving the log in an inconsistent or partially unsafe state. Group 1 deviated from this expected behavior in the tested batch case. We therefore record it as a behavioral issue, separate from the cryptographic vulnerability discussed below.

5.5 Reconnaissance and Code Review Methodology

We reviewed the implementation with the security goals from the project statement in mind: confidentiality of the log contents without the token, integrity against unauthorized modification, authenticity of appends and reads, and correct handling of invalid gallery-state transitions.

- **Log format:** Each log file begins with a fixed header containing a magic value (BLOG), a version byte, and a random salt. Each appended record stores:

  ```
  length || IV || AES-GCM(ciphertext)
  ```

  The plaintext encrypted inside each record contains:

```
prevHash || timestamp || payload
```

where `prevHash` is the SHA-256 hash of the previous raw record.

- **Key derivation and authentication:** The implementation derives a 256-bit AES key from the provided token using PBKDF2-HMAC-SHA256 with 120000 iterations and a per-log random salt. AES-GCM authentication tags are used to verify ciphertext integrity.
- **Encryption design:** The implementation uses AES-256-GCM for authenticated encryption. However, the IV generation is critically flawed. The IV is deterministically generated from the log filename using:

```
SecureRandom vulnerableRandom = new SecureRandom();
vulnerableRandom.setSeed(logPath.getBytes(...));
vulnerableRandom.nextBytes(iv);
```

  As a result, every record appended to the same log file reuses the exact same AES-GCM IV under the same encryption key. AES-GCM nonce reuse is cryptographically catastrophic and breaks confidentiality guarantees.
- **Integrity checks:** The implementation validates a SHA-256 hash chain linking consecutive records. Any modification, deletion, truncation, or byte tampering inside a record causes AES-GCM authentication or hash-chain mismatch detection.
- **Input validation:** The implementation validates timestamps, room identifiers, state transitions, duplicate flags, malformed arguments, and ordering constraints using a dedicated state machine.
- **Filename handling:** During behavioral testing, we observed that the log filename argument is passed directly to the Java file I/O layer without path sanitization. This allows directory-traversal sequences (e.g., `../../`) to escape the intended working directory.
- **Record-size analysis:** The encrypted record structure uses fixed-size cryptographic fields (12-byte IV, 32-byte previous hash, 8-byte timestamp) plus a variable-length payload that depends on the employee/guest name and optional room number. Because no padding is applied to normalize record sizes, the total ciphertext length directly leaks information about the plaintext payload size.
- **Rollback-susceptibility analysis:** We examined whether the implementation could detect substitution of the log file with an older valid copy. The hash-chain design only validates that each record correctly links to its immediate predecessor; it does not maintain any global counter, cumulative signature, or out-of-band checkpoint that would detect truncation or rollback of the log tail. Consequently, replacing the current log with an older version produces a file that passes all internal integrity checks, and subsequent appends simply extend the chain from the truncated point.

5.6 Attack Summary

ID	Type	Status	Impact
G1-A1	Confidentiality	Exploited	AES-GCM nonce reuse leaks plaintext relationships
G1-A2	Confidentiality	Exploited	Predictable record sizes leak entry count and name-length information
G1-A3	Integrity	Exploited	Path traversal in log filename allows arbitrary file write/read
G1-A4	Integrity	Exploited	Undetected log rollback allows selective deletion of history; continued read/write accepted
G1-A5	Integrity	Not exploited	Potential forgery conditions due to nonce reuse

Table 3: Summary of findings for Group 1

5.7 Finding G1-A1: AES-GCM Nonce Reuse Due to Deterministic IV Generation

5.7.1 High-Level Description

The implementation deterministically derives the AES-GCM IV from the log filename instead of generating a fresh random nonce for every encrypted record. Consequently, all records stored in the same log file reuse the same IV under the same AES key.

AES-GCM requires nonce uniqueness for security. Reusing a nonce with the same key completely breaks confidentiality and may enable forgery attacks. An attacker without the token can compare ciphertexts from different records and derive relationships between plaintext contents.

5.7.2 Root Cause

The vulnerability originates in the `CryptoService.randomIv()` method:

```
static byte[] randomIv(String logPath) {
    byte[] iv = new byte[Constants.GCM_IV_BYTES];
    SecureRandom vulnerableRandom = new SecureRandom();
    vulnerableRandom.setSeed(logPath.getBytes(StandardCharsets.UTF_8));
    vulnerableRandom.nextBytes(iv);
    return iv;
}
```

Because the seed depends only on the log path, every append operation targeting the same file generates the same IV. The resulting AES-GCM nonce reuse violates the security assumptions of GCM mode.

5.7.3 Exploit Preconditions

- The attacker can access the encrypted log file but does not know the token.
- The attacker can observe multiple encrypted records stored in the same log.
- The attacker does not need to modify the file or interact with the legitimate user.

5.7.4 Reproduction Steps

```
# 1. Create a log with multiple records
rm -f nonceReuse.log

./logappend -T 1 -K secret -A -E Alice nonceReuse.log
./logappend -T 2 -K secret -A -G Bob nonceReuse.log
./logappend -T 3 -K secret -A -E Alice -R 1 nonceReuse.log

# 2. Observe the binary log contents
xxd nonceReuse.log

# 3. Compare encrypted records
# The IV prefix of every encrypted record is identical.
```

5.7.5 Expected Output on a Correct Implementation

```
Each encrypted record should use a unique random IV.
```

5.7.6 Observed Output on Group 1

```
The IV bytes at the beginning of each encrypted record are identical across
all records stored in the same log file.
```

5.7.7 Security Impact

The attack is security-relevant because AES-GCM nonce reuse completely invalidates the confidentiality guarantees of the encrypted log. Since multiple plaintexts are encrypted using the same keystream, an attacker can compute XOR relationships between plaintext records without knowing the encryption key.

The plaintext structure is highly predictable because every record contains:

- a fixed-size previous hash field,
- a timestamp,
- structured payload fields,
- repeated names and room identifiers.

This predictability enables statistical plaintext recovery and cryptographic analysis of encrypted records. Our `exploit.py` script demonstrates that even without the token, an attacker can parse the log file, enumerate all records, detect IV reuse, and infer approximate payload sizes—information that significantly narrows the space of possible plaintexts and facilitates keystream-recovery attacks.

Under certain conditions, AES-GCM nonce reuse may also enable message forgery attacks.

5.7.8 Attack Code

We developed a Python script named `exploit.py` that parses the encrypted log file, extracts the IV and ciphertext of every record, and performs automated analysis of the nonce-reuse condition. The script produces a detailed per-record breakdown including offset, IV value, ciphertext length, inferred plaintext length, and event-type classification based on payload-size heuristics. It also performs an IV-reuse check across all records and flags every pair that shares the same nonce.

The following output demonstrates `exploit.py` running against a Group 1 log file containing six records:

```
$ py exploit.py logfile5
[+] Salt (hex): be8ca1bba9e15644ecd7f749bbc11079
[+] Total file size: 531 bytes
[+] Total records: 6

=== IV reuse check ===
[!!!!] CRITICAL: Record 2 and Record 3 share IV 42412d1d0fabc94c59072d94
       AES-GCM key+IV pair reused - keystream XOR attack possible
[!!!!] CRITICAL: Record 3 and Record 4 share IV 42412d1d0fabc94c59072d94
       AES-GCM key+IV pair reused - keystream XOR attack possible
[!!!!] CRITICAL: Record 4 and Record 5 share IV 42412d1d0fabc94c59072d94
       AES-GCM key+IV pair reused - keystream XOR attack possible
[!!!!] CRITICAL: Record 5 and Record 6 share IV 42412d1d0fabc94c59072d94
       AES-GCM key+IV pair reused - keystream XOR attack possible
```

As the script output confirms, Records 2–6 all reuse the exact same IV `42412d1d0fabc94c59072d94`. Only Record 1 uses a different IV (`4aadcdfb6199a5a63b577c85`), which corresponds to the initial log creation. This pattern is consistent with the deterministic IV generation: the first record is created when the log file does not yet exist, while all subsequent records are appended to an existing file and therefore receive the same filename-derived seed.

In addition to detecting IV reuse, the script infers structural information from each record's ciphertext length:

- Record 1: 68B ciphertext → 52B plaintext → inferred as GALLERY event with 5-character name or ROOM event with 1-character name.
- Record 2: 72B ciphertext → 56B plaintext → inferred as GALLERY event with 9-character name or ROOM event with 5-character name.
- Record 3: 67B ciphertext → 51B plaintext → inferred as GALLERY event with 4-character name or ROOM event with 0-character name.

- Record 4: 69B ciphertext → 53B plaintext → inferred as GALLERY event with 6-character name or ROOM event with 2-character name.
- Record 5: 67B ciphertext → 51B plaintext → inferred as GALLERY event with 4-character name or ROOM event with 0-character name.
- Record 6: 71B ciphertext → 55B plaintext → inferred as GALLERY event with 8-character name or ROOM event with 4-character name.

These inferences are possible because the fixed overhead (32-byte `prevHash` + 8-byte timestamp + 16-byte GCM tag + 12-byte IV + 4-byte length) is known, so the remaining bytes directly reveal the payload size and therefore approximate name lengths.

```
PS C:\Users\Utilizador\Documents\GitHub\SRSD\G01\SRSD_GalleryLog> py exploit.py logfile5
[+] Salt (hex): be8ca1bba9e15644ecd7f749bbc11079
[+] Total file size: 531 bytes

--- Record 1 (offset=21, body=80B) ---
  IV:             4aadcdfb6199a5a63b577c85
  Ciphertext:     68B  (plaintext: 52B)
  Event type:     GALLERY (5 char name) OR ROOM (1 char name)
  [ambiguous - check adjacent records for context]
--- Record 2 (offset=105, body=84B) ---
  IV:             42412d1d0fabc94c59072d94
  Ciphertext:     72B  (plaintext: 56B)
  Event type:     GALLERY (9 char name) OR ROOM (5 char name)
  [ambiguous - check adjacent records for context]
--- Record 3 (offset=193, body=79B) ---
  IV:             42412d1d0fabc94c59072d94
  Ciphertext:     67B  (plaintext: 51B)
  Event type:     GALLERY (4 char name) OR ROOM (0 char name)
  [ambiguous - check adjacent records for context]
--- Record 4 (offset=276, body=81B) ---
  IV:             42412d1d0fabc94c59072d94
  Ciphertext:     69B  (plaintext: 53B)
  Event type:     GALLERY (6 char name) OR ROOM (2 char name)
  [ambiguous - check adjacent records for context]
--- Record 5 (offset=361, body=79B) ---
  IV:             42412d1d0fabc94c59072d94
  Ciphertext:     67B  (plaintext: 51B)
  Event type:     GALLERY (4 char name) OR ROOM (0 char name)
  [ambiguous - check adjacent records for context]
--- Record 6 (offset=444, body=83B) ---
  IV:             42412d1d0fabc94c59072d94
  Ciphertext:     71B  (plaintext: 55B)
  Event type:     GALLERY (8 char name) OR ROOM (4 char name)
  [ambiguous - check adjacent records for context]

[+] Total records: 6

=== IV reuse check ===
[!!!] CRITICAL: Record 2 and Record 3 share IV 42412d1d0fabc94c59072d94
      AES-GCM key+IV pair reused - keystream XOR attack possible
[!!!] CRITICAL: Record 3 and Record 4 share IV 42412d1d0fabc94c59072d94
      AES-GCM key+IV pair reused - keystream XOR attack possible
[!!!] CRITICAL: Record 4 and Record 5 share IV 42412d1d0fabc94c59072d94
      AES-GCM key+IV pair reused - keystream XOR attack possible
[!!!] CRITICAL: Record 5 and Record 6 share IV 42412d1d0fabc94c59072d94
      AES-GCM key+IV pair reused - keystream XOR attack possible
```

Figure 2: Output of `exploit.py` demonstrating AES-GCM IV reuse across six records in a Group 1 log file, with per-record ciphertext analysis and structural inference.

The script will be submitted together with the report as `attacks/group1/exploit.py`.

5.8 Finding G1-A2: Information Disclosure via Predictable Record Sizes

5.8.1 High-Level Description

The encrypted log file uses a fixed-size record structure with no padding on the plaintext payload. Each record contains a 12-byte IV, a 32-byte previous-hash field, an 8-byte timestamp, and a variable-length payload that depends on the employee/guest name length and whether a room number is present. Because the ciphertext length is exactly the plaintext length plus constant overhead, an attacker who observes the encrypted log file—or tracks its size across multiple versions—can infer the number of entries it contains and approximate the length of individual names and room identifiers.

This is a confidentiality violation: the project specification states that an attacker without the token should not be able to infer private log contents from the file representation alone. The length side-channel directly leaks structural information about the plaintext.

5.8.2 Root Cause

The vulnerability originates in the record serialization and encryption design. The implementation concatenates fixed-width cryptographic metadata with the variable-length payload and encrypts the result using AES-GCM without first padding the plaintext to a uniform block size:

```
record = length || IV || AES-GCM( prevHash || timestamp || payload )
```

where `prevHash` is 32 bytes, `timestamp` is 8 bytes, and `payload` varies. AES-GCM preserves plaintext length in the ciphertext, so the total record size is:

$$\text{record size} = 4 + 12 + 16 + (32 + 8 + \text{len(payload)})$$

The 16-byte term is the GCM authentication tag. Consequently, a difference of one character in an employee name produces a one-byte difference in the encrypted file size.

5.8.3 Exploit Preconditions

- The attacker can access the encrypted log file (either a single snapshot or multiple versions over time).
- The attacker does not know the authentication token.
- The attacker may optionally have access to an earlier version of the same log to perform differential size analysis.

5.8.4 Reproduction Steps

The following commands demonstrate the length side-channel:

```
# 1. Create two logs that differ only in employee name length
./logappend -T 1 -K secret -A -E Alice shortName.log
./logappend -T 1 -K secret -A -E Al   shorterName.log

# 2. Compare file sizes
ls -l shortName.log shorterName.log
# Observed: shorterName.log is exactly 3 bytes smaller
# (difference between "Alice" (5 bytes) and "Al" (2 bytes))

# 3. Create logs with and without room numbers to measure payload difference
./logappend -T 1 -K secret -A -E Alice noRoom.log
./logappend -T 1 -K secret -A -E Alice -R 5 withRoom.log

# 4. Compare sizes
ls -l noRoom.log withRoom.log
# The size difference reveals the presence and length of the room field
```

5.8.5 Expected Output on a Correct Implementation

A correct implementation should pad each plaintext record to a fixed block size before encryption, or apply a padding scheme that hides payload-length variation. Under such a design, the encrypted file size should reveal only an approximate upper bound on the number of entries, not exact per-record lengths or name sizes.

5.8.6 Observed Output on Group 1

Group 1 produces encrypted logs whose file sizes vary predictably with plaintext content:

- Changing the employee name from "Alice" (5 bytes) to "Al" (2 bytes) reduces the total file size by exactly 3 bytes.
- Adding a room number to the payload increases the file size by the exact number of bytes required to encode that room field.

This confirms that the ciphertext length directly mirrors the plaintext payload length.

5.8.7 Security Impact

This vulnerability is a confidentiality break with the following consequences:

- **Entry-count enumeration:** By dividing the total encrypted file size by the known per-record overhead, an attacker can determine the exact number of log entries without the token.
- **Name-length inference:** By comparing two versions of the same log (or logs created with similar parameters but different names), an attacker can infer the length of employee and guest names.
- **Activity-pattern analysis:** Observing file-size changes over time reveals when new entries are added and whether those entries include room transitions (which add extra payload bytes).
- **Reduced brute-force space:** Knowing name lengths significantly narrows the search space for dictionary attacks against employee or guest identities.

5.8.8 Attack Code

No custom exploit code is required. The vulnerability is demonstrated using standard filesystem commands:

```
ls -l *.log
```

Differential analysis can be performed by comparing the byte sizes of two encrypted logs created with systematically varied parameters.

5.9 Finding G1-A3: Path Traversal via Unsanitized Log Filename

5.9.1 High-Level Description

The `logappend` and `logread` programs accept a log filename as their final positional argument and pass it directly to the Java file I/O layer without sanitizing directory-traversal sequences such as `../`. This allows an attacker to write encrypted log files to arbitrary locations on the filesystem and subsequently read them back using equivalent traversal paths. The vulnerability violates the integrity and confidentiality boundaries of the application: logs can be stored outside the intended working directory, potentially overwriting existing files or exposing sensitive data in attacker-controlled locations.

5.9.2 Root Cause

The vulnerability originates in the absence of input validation on the log filename parameter. Both `logappend` and `logread` treat the filename argument as a raw path string and open it via standard Java file constructors. No checks restrict the path to the current working directory or reject parent-directory references (`..`).

5.9.3 Exploit Preconditions

- The attacker can invoke `logappend` or `logread` with an arbitrary filename argument.
- The attacker knows or can guess a relative path that reaches a target directory outside the intended log directory.
- The operating-system user running the gallery-log programs has write (for `logappend`) or read (for `logread`) permissions in the target directory.

5.9.4 Reproduction Steps

The following commands reproduce the path-traversal vulnerability on Group 1:

```
# 1. Write a log file outside the program directory using traversal
./logappend -T 1 -K secret -A -E Alice \
    ../../quicktest/logfiletest

# 2. Verify the file was created in the escaped directory
ls -l ../../quicktest/
# Output shows: logfiletest

# 3. Read the escaped log file back using an equivalent traversal
./logread -K secret -S ../../quicktest/logfiletest
# Output: Alice
```

The same traversal works from different relative depths, allowing an attacker to place or retrieve logs anywhere the process has filesystem access.

5.9.5 Expected Output on a Correct Implementation

A correct implementation should sanitize the filename argument to prevent directory traversal. Acceptable defenses include:

- Rejecting filenames that contain `..` or path-separator characters.
- Resolving the path against the intended log directory and verifying the resolved path is still within that directory.
- Using a chroot-like or sandboxed file-access mechanism.

Under any of these defenses, the commands above should produce an error such as "invalid filename" or "access denied" rather than creating or reading a file outside the intended directory.

5.9.6 Observed Output on Group 1

Group 1 accepts the traversal sequence and creates the log file at the attacker-specified location:

```
# logappend succeeds and writes ../../quicktest/logfiletest
# logread succeeds and returns the stored state from that location
```

No error is raised, and the file is treated as a valid encrypted log.

5.9.7 Security Impact

This vulnerability is an integrity and confidentiality break with the following consequences:

- **Arbitrary file write:** An attacker can cause `logappend` to create or overwrite files in sensitive directories (e.g., user home, temporary directories, or even system paths if the process runs with elevated privileges). This could be used to plant malicious data, exhaust disk quotas, or corrupt unrelated files.
- **Arbitrary file read:** An attacker can cause `logread` to attempt decryption of any file on the filesystem. While AES-GCM decryption will fail for non-log files, the file content is still read into memory, which may expose unrelated data if the file happens to have a compatible structure.
- **Log relocation and evasion:** An attacker can store logs in hidden or unexpected locations, making forensic analysis and audit trails harder to follow.
- **Information disclosure via error channels:** If the attacker points `logread` at an existing non-log file, the resulting decryption or parsing errors may leak information about the file's contents or structure.

5.9.8 Attack Code

No custom exploit code is required. The vulnerability is reproduced using standard shell commands with directory-traversal sequences in the filename argument:

```
./logappend -T 1 -K secret -A -E Alice ../../target/escaped.log
./logread -K secret -S ../../target/escaped.log
```

5.10 Finding G1-A4: Undetected Log Rollback via Older Version Substitution

5.10.1 High-Level Description

The integrity mechanism of Group 1 uses a per-record hash chain that links each entry to its immediate predecessor. While this prevents reordering and tampering of individual records, it does not protect against rollback attacks: an attacker can replace the current log file with an older copy of the same log, and the implementation accepts the substituted file without detecting that intermediate entries have been removed. Subsequent read and append operations proceed normally on the rolled-back state, permanently erasing the deleted history and allowing new entries to be appended to the truncated chain as if the missing records never existed.

This is a critical integrity failure because the fundamental purpose of a secure audit log is append-only immutability: once an event is recorded, it must not be possible to remove it without detection.

5.10.2 Root Cause

The vulnerability originates in the absence of a global anti-rollback mechanism. The implementation validates integrity by checking that each record's `prevHash` matches the SHA-256 hash of the previous raw record. This chain-of-hashes design ensures that individual records cannot be modified or reordered, but it does not detect truncation or replacement of the log tail because:

- There is no monotonic entry counter in the file header that is checked on every read or append.
- There is no cumulative signature or Merkle-tree root that binds the entire log history.
- The `logread` and `logappend` programs do not maintain any out-of-band state (e.g., a trusted counter or checkpoint) against which to verify the log's current length or latest hash.

Consequently, if an older version of the log is substituted, the hash chain is still internally consistent up to that older version's last record, and new appends simply extend the chain from that truncated point.

5.10.3 Exploit Preconditions

- The attacker has access to the log file at two different points in time (e.g., by copying it after an initial set of entries and again after further entries have been added).

- The attacker can replace the current log file with the older copy (e.g., via filesystem copy, restore from backup, or symlink manipulation).
- The attacker has a valid token for the log (or the rollback is performed by a legitimate but malicious user).

5.10.4 Reproduction Steps

The following commands demonstrate the rollback vulnerability on Group 1:

```
# 1. Create an initial log and add the first employee
./logappend -T 1 -K secret -A -E Alice logfile6

# 2. Save a copy of the log at this point in time
cp logfile6 logfile6_backup

# 3. Continue appending more entries to the original log
./logappend -T 2 -K secret -A -E Adam logfile6
./logappend -T 3 -K secret -A -E Adam -R 1 logfile6

# 4. Verify the current state before rollback
./logread -K secret -S logfile6
# Expected output:
# Adam,Alice
#
# 1: Adam

# 5. Roll back: replace the current log with the older backup
cp logfile6_backup logfile6

# 6. Read the rolled-back log - accepted without any error
./logread -K secret -S logfile6
# Observed output:
# Alice
# (Adam's arrival and room entry are completely gone)

# 7. Continue appending to the rolled-back log
./logappend -T 4 -K secret -A -E Eve logfile6

# 8. Read again - new append accepted, history permanently altered
./logread -K secret -S logfile6
# Observed output:
# Alice,Eve
# (Adam never existed in the log; the audit trail is silently rewritten)
```

5.10.5 Expected Output on a Correct Implementation

A correct implementation must detect that the log file has been rolled back to an older state and reject operations on the truncated file. Acceptable defenses include:

- A monotonic entry counter stored in the file header and verified on every `logread` and `logappend` invocation.
- A cumulative hash or Merkle-tree root signed with the token-derived key that covers the entire log.
- Rejection of any log whose total size or record count is smaller than a previously observed value.
- A trusted side-channel or sealed state that records the latest valid log hash and rejects older versions.

Under any of these defenses, substituting an older version of the log should produce an error such as "integrity violation" or "log rolled back" rather than silently accepting the truncated history.

5.10.6 Observed Output on Group 1

Group 1 accepts the older log file without detecting the rollback:

```
# After rollback to backup:
./logread -K secret -S logfile6
# Output: Alice (intermediate entries erased)

# New append after rollback:
./logappend -T 4 -K secret -A -E Eve logfile6
# Output: accepted (no error)

# Final state:
./logread -K secret -S logfile6
# Output: Alice,Eve (Adam's entries permanently removed from history)
```

The hash chain validates because each record only checks its immediate predecessor, and the predecessor of the new Eve entry is the last record of the rolled-back Alice-only log.

5.10.7 Security Impact

This vulnerability is a severe integrity break with the following consequences:

- **Selective history deletion:** An attacker can erase any subset of log entries by restoring a backup taken before those entries were added. This violates the append-only audit property fundamental to secure logging.
- **Audit evasion:** A malicious insider can cover their tracks by rolling back the log to a state before their unauthorized actions were recorded, then continuing normal operation.
- **Evidence tampering:** In a forensic or compliance context, the ability to substitute an older log version without detection undermines the non-repudiation guarantees of the system.
- **Chain-validated deception:** Because new appends after a rollback still produce valid hash chains, the tampered log appears cryptographically intact to casual inspection, making the attack difficult to detect post-hoc.

5.10.8 Attack Code

No custom exploit code is required. The vulnerability is reproduced using standard filesystem copy commands:

```
cp logfile6_backup logfile6
./logread -K secret -S logfile6
./logappend -T 4 -K secret -A -E Eve logfile6
```

5.11 Finding G1-A5: Potential Integrity Forgery via AES-GCM Nonce Reuse

5.11.1 Description

During the analysis, we identified that the AES-GCM nonce reuse vulnerability may also enable integrity attacks or ciphertext forgery attempts. Although we did not complete a practical forgery exploit during the Break-it phase, the cryptographic misuse significantly weakens the authenticated encryption guarantees of AES-GCM.

Because the implementation correctly validates hash chains and authentication tags, we were unable to produce a complete unauthorized append accepted by `logread`. Nevertheless, nonce reuse under AES-GCM is considered a severe cryptographic failure.

5.12 Tests Attempted but Found Secure

We also tested several attack ideas against Group 1 that did not result in successful exploits:

- **Record reordering attack:** Reordering existing records broke the previous-record hash chain and was rejected.
- **Direct ciphertext tampering:** Modifying bytes inside existing records caused AES-GCM authentication or chain validation to fail.
- **Wrong-token operations:** Reads and appends with an incorrect token were rejected safely in the tested cases.
- **Invalid gallery transitions:** Duplicate arrivals, leaving without entering, and invalid room transitions were rejected by the state machine.
- **Malformed length fields:** Corrupted record length fields did not lead to an exploitable crash or uncontrolled memory allocation.

5.13 Vulnerabilities Considered but Not Exploited

- **Record reordering attack:** We tested whether records could be reordered while preserving the hash chain. The implementation correctly rejected reordered logs due to previous-record hash validation.
- **Malformed length fields:** We investigated whether corrupted record length fields could trigger parser crashes or uncontrolled memory allocation. The implementation included explicit upper bounds on record size and rejected malformed lengths safely.

5.14 Conclusion for Group 1

Group 1 contains multiple security-relevant vulnerabilities. The strongest finding is the catastrophic AES-GCM nonce reuse vulnerability (G1-A1) caused by deterministic IV generation based solely on the log filename. This fundamentally breaks confidentiality guarantees for encrypted records, allowing an attacker without the token to derive XOR relationships between plaintexts.

In addition, we identified three further security failures. The encrypted log file leaks structural information through predictable record sizes (G1-A2): because the payload is not padded before encryption, file-length differences reveal the number of entries, name lengths, and the presence of room fields. The log filename argument is vulnerable to path traversal (G1-A3), allowing an attacker to write logs to arbitrary filesystem locations and read them back from escaped directories, breaking integrity and confidentiality boundaries. Most critically for log integrity, the hash-chain design fails to detect rollback attacks (G1-A4): substituting an older version of the log file goes completely undetected, allowing an attacker to selectively erase history and continue appending new entries to the truncated chain as if the deleted records never existed. This violates the fundamental append-only immutability property required of any secure audit log.

Although the implementation correctly enforced authentication, state validation, and hash-chain integrity against individual record tampering in several areas, the combination of cryptographic misuse (nonce reuse), length side-channels, missing path sanitization, and rollback susceptibility constitute severe design errors.

6 Implementation G21: Group 21

6.1 Target Summary

- **Assigned implementation:** Group 21.
- **Language and build system:** C with Makefile and OpenSSL.
- **Programs analyzed:** `logappend` and `logread`.
- **Behavioral status:** Passed most functional and security-behavior tests.
- **Main result:** The implementation behaved safely in the tested invalid-action cases, but it reported some safe rejections with the wrong return code. It also showed the same room-history duplication issue observed in Group 1. **Most critically, we identified two security vulnerabilities: a race-condition integrity vulnerability that allows concurrent `logappend` processes to corrupt the log state, leaving it in an inconsistent condition that permanently rejects all subsequent legitimate operations, and weak PBKDF2 parameters that make offline token recovery practical.**

6.2 How We Built and Ran the Implementation

Observed result: The implementation compiled successfully using GCC and OpenSSL on Ubuntu Linux. Both `logappend` and `logread` executed correctly during normal operation. No special setup beyond OpenSSL development libraries was required.

```
cd group21
make
./logappend -T 1 -K testToken -E Alice -A test.log
./logread -K testToken -S test.log
```

6.3 Behavioral Test Results

Table 4 summarizes the Group 21 execution of the shared test suite described in Section 4. The full raw output is omitted from the main text because it mostly consists of successful setup commands and repeated expected outputs.

Test area	Result	Relevant observation
Basic gallery flow	Pass	Employee and guest arrivals/departures were reflected correctly by `-S`.
Room transitions	Pass	Room entry and exit transitions were enforced safely.
State and intersection queries	Pass	`-S` and `-I` returned the expected state for the tested scenarios.
Room history query	Minor issue	Revisited rooms were printed again, e.g., `3,7,3` instead of the deduplicated `3,7`.
Invalid actions	Safe with return-code issue	Unsafe operations were not applied, but some were signalled with an incorrect return code.
Wrong token and tampering	Pass	The tested unsafe operations did not expose protected data or produce an accepted corrupted state.
Batch mode	Pass	Valid lines were processed and intentionally invalid lines were handled safely.

Table 4: Condensed behavioral results for Group 21

6.4 Behavioral Findings

6.4.1 Room History Duplication

The `-R` query includes repeated room visits in the output. For example, if a person visits room 3, later room 7, and then room 3 again, the implementation prints `3,7,3`. Our interpretation of the expected behavior is that room history should list each visited room once, in first-visit order, i.e., `3,7`. This is a correctness issue rather than a direct security vulnerability because it does not let an attacker read, modify, or forge protected log contents.

6.4.2 Incorrect Return Codes for Safe Rejections

Some invalid operations were rejected safely but did not use the expected error signalling convention. In these cases, the important security property still held: the invalid action was not committed to the log. Therefore, we classify these as compatibility/correctness issues, not successful Break-it attacks.

6.5 Reconnaissance and Code Review Methodology

Following the behavioral tests, we reviewed the implementation with emphasis on cryptographic design, state consistency, log integrity enforcement, and malformed-input handling.

- **Log format:** The log file consists of a plaintext salt header followed by encrypted metadata and append-only encrypted entries.
- **Key derivation and authentication:** Authentication tokens are converted into AES keys using PBKDF2-HMAC-SHA256.
- **Encryption design:** Entries are encrypted using AES-GCM with per-entry IVs and authentication tags.
- **Integrity checks:** The implementation uses authenticated encryption and chaining metadata to detect malformed or reordered records.
- **Input validation:** Names, timestamps, room identifiers, duplicate flags, and gallery-state transitions are validated before committing new entries.

Cryptographic analysis: We examined the key-derivation mechanism and identified that PBKDF2-HMAC-SHA256 is configured with only **1 iteration**, far below modern secure standards. This dramatically reduces the computational cost of offline brute-force attacks against the authentication token.

Concurrency analysis: During code review, we examined how the implementation handles concurrent `logappend` invocations targeting the same log file. We observed that the program opens the log file, reads its current state to validate transitions (e.g., checking whether an employee is already in a room, whether a timestamp is valid, and whether the number of entries is consistent), and then writes the new encrypted record. However, **there is no file locking or atomic read-modify-write primitive protecting the validation-and-append sequence**. This creates a Time-of-Check to Time-of-Use (TOCTOU) race window: two concurrent processes can both read the same log state, both decide their respective operations are valid, and both write, producing an internally inconsistent log.

6.6 Attack Summary

ID	Type	Status	Impact
G21-A2	Confidentiality	Exploited	Offline brute-force attack possible due to PBKDF2 configured with 1 iteration
G21-A1	Integrity	Exploited	Race-condition TOCTOU allows concurrent appends to corrupt log state, causing permanent denial of service
G21-B1	Correctness	Reproduced	Duplicate rooms in `-R` history output
G21-B2	Compatibility	Reproduced	Some safe rejections use the wrong return code

Table 5: Summary of Group 21 findings

6.7 Finding G21-A1: Race-Condition Integrity Violation via Concurrent Logappend

6.7.1 High-Level Description

The implementation does not protect the read-check-write sequence of `logappend` with file locking or any other mutual-exclusion mechanism. When two `logappend` processes execute concurrently against the same log file, they can both read the current log state before either writes, bypass the single-room-occupancy invariant, and append mutually inconsistent records. The resulting log becomes permanently corrupted: subsequent legitimate operations (including valid appends with correct tokens and timestamps) are rejected with "integrity violation," producing a complete denial of service against the log.

6.7.2 Root Cause

The vulnerability originates in the absence of atomicity between state validation and state modification in `logappend`. The program performs the following non-atomic sequence:

1. Open the log file and read the current state (number of entries, active room assignments, latest timestamps).
2. Validate the proposed new entry against the read state (e.g., check that the employee is not already in another room, that the timestamp increases monotonically, and that the entry count is consistent).
3. Encrypt and write the new record to the log file.

Because steps 1–3 are not wrapped in a lock, two concurrent processes can interleave as follows:

1. Process A reads state S.
2. Process B reads the same state S.
3. Process A validates its operation against S and decides it is safe.
4. Process B validates its operation against S and also decides it is safe.
5. Process A writes record R_A.
6. Process B writes record R_B.

Both operations were validated against the same stale state S, so invariants that depend on sequential state updates (e.g., "an employee can only be in one room at a time") are violated.

6.7.3 Exploit Preconditions

- The attacker has a valid token for the target log file (or is a legitimate user).
- The attacker can launch two or more `logappend` processes concurrently, for example by backgrounding them in a shell subscript or using parallel execution tools.
- The concurrent operations must target the same log file and must involve state-dependent invariants (e.g., room occupancy, timestamp ordering, or entry-count consistency).

6.7.4 Reproduction Steps

The following shell commands reproduce the vulnerability on Group 21:

```
# 1. Create a clean log and add the first valid entry
./logappend -T 1 -K raceToken -E Alice -A test4

# 2. Attempt a duplicate arrival (should be rejected)
./logappend -T 1 -K raceToken -E Alice -A test4
# Output: invalid

# 3. Launch two concurrent room-entry commands for Alice
(
    ./logappend -T 2 -K raceToken -E Alice -A -R 10 test4 &
    ./logappend -T 3 -K raceToken -E Alice -A -R 20 test4 &
    wait
)

# 4. Read the corrupted log state
./logread -K raceToken -S test4
# Output:
# Alice
#
# 20:Alice
# (Alice appears to be in room 20, but never left room 10)

# 5. Any subsequent legitimate append fails
./logappend -T 4 -K raceToken -E Alice -A -R 30 test4
# Output: integrity violation

# 6. Even appending a different employee fails
./logappend -T 3 -K raceToken -E Adam -A test4
# Output: integrity violation
./logappend -T 2 -K raceToken -E Adam -A test4
# Output: integrity violation
./logappend -T 4 -K raceToken -E Adam -A test4
# Output: integrity violation
./logappend -T 5 -K raceToken -E Adam -A test4
# Output: integrity violation
./logappend -T 10 -K raceToken -E Adam -A test4
# Output: integrity violation

# 7. The log remains permanently corrupted
./logread -K raceToken -S test4
# Output:
# Alice
#
# 20:Alice
```

6.7.5 Expected Output on a Correct Implementation

A correct implementation must serialize concurrent `logappend` operations so that each append is validated against the state produced by all previously committed appends. In the reproduction above, one of the two concurrent room entries should be rejected (e.g., the second one chronologically, or whichever loses the race), and the log should remain in a consistent state:

```
Alice
10:Alice
```

Subsequent legitimate appends with increasing timestamps should succeed.

6.7.6 Observed Output on Group 21

Group 21 accepts both concurrent room entries, leaving Alice simultaneously in room 20 and implicitly in room 10 (since she never left it). The `-S` query shows:

```
Alice

20:Alice
```

After this corruption, **every** subsequent `logappend`—including valid operations with correct tokens, increasing timestamps, and different employees—returns "integrity violation." The log file is permanently unusable.

6.7.7 Security Impact

This attack is a security-relevant integrity violation with denial-of-service impact:

- **Integrity violation:** The log accepts an impossible state (Alice in two rooms simultaneously) that violates the gallery-transition rules. The integrity of the state machine is broken.
- **Denial of service:** Once corrupted, the log rejects all further legitimate operations. An attacker with token access can permanently disable the log by executing a single concurrent double-append, preventing any future auditing or state tracking.
- **Audit failure:** The primary purpose of a secure gallery log is to maintain an accurate, tamper-evident audit trail. A corrupted log that rejects all future writes defeats this purpose entirely.

6.7.8 Attack Code

The attack requires no custom exploit code. It is reproduced using standard shell backgrounding:

```
(
    ./logappend -T 2 -K raceToken -E Alice -A -R 10 test4 &
    ./logappend -T 3 -K raceToken -E Alice -A -R 20 test4 &
    wait
)
```

6.8 Tests Attempted but Found Secure

Besides the behavioral issues listed above, we also tried several attacks that did *not* lead to successful security breaks in Group 21:

- **Invalid state transitions:** Attempts to leave without entering, enter two rooms at the same time, or perform duplicate arrivals were rejected without committing unsafe entries.
- **Wrong-token append attempts:** Appends using an incorrect token did not modify the protected log state.
- **Direct log tampering:** The tested byte-modification scenarios did not produce an accepted corrupted state.
- **Batch invalid-line handling:** Invalid batch lines were handled safely, even when the reported return code did not always match the expected convention.
- **Malformed input arguments:** Invalid names, conflicting flags, and invalid room identifiers were rejected before affecting the log.

6.9 Conclusion for Group 21

Group 21 passed most of the shared behavioral suite. The main observed problems were not successful security attacks: room-history output duplicates revisited rooms, and some invalid operations are signalled with an incorrect return code even though the actions are safely rejected. Because the unsafe actions were not committed, we do not classify those return-code mismatches as integrity violations.

However, **the weak PBKDF2 configuration (G21-A2) is the most severe security-relevant finding.** The implementation uses PBKDF2-HMAC-SHA256 with only a single iteration, making offline dictionary and brute-force attacks practical. Using a standard Kali Linux wordlist and our provided `crack.py` script, we successfully recovered authentication tokens from protected logs. This constitutes a critical confidentiality violation, as knowledge of the token also enables unauthorized appends and further integrity compromise.

We additionally identified a serious race-condition integrity vulnerability (G21-A1). The absence of file locking or atomic read-modify-write primitives allows concurrent `logappend` processes to corrupt the log state, producing an impossible room-occupancy condition and permanently denying service to all subsequent legitimate operations. This constitutes both an integrity violation (acceptance of an invalid state transition) and a denial-of-service attack against the audit log.

7 Overall Conclusion

Both implementations passed most of the shared behavioral suite. The common behavioral issue was room-history duplication in `-R`: revisited rooms were printed multiple times instead of being deduplicated. Group 1 additionally showed a batch-mode deviation, while Group 21 sometimes signalled safe rejections with an incorrect return code.

We classify these demo findings as correctness or compatibility issues unless the unsafe action is actually committed or protected data is exposed.

Security-relevant findings: Group 1 suffers from four major vulnerabilities: (1) a catastrophic AES-GCM nonce reuse vulnerability (G1-A1) caused by deterministic IV generation, which completely breaks the confidentiality guarantees of the encrypted log; (2) an information-disclosure side-channel (G1-A2) via predictable encrypted record sizes that leaks entry counts and name lengths; (3) a path-traversal vulnerability (G1-A3) in the log filename argument that allows arbitrary file write and read outside the intended directory; and (4) an undetected log-rollback vulnerability (G1-A4) in which substituting an older version of the log file goes completely undetected, allowing an attacker to selectively erase history and continue appending new entries to the truncated chain as if the deleted records never existed. This violates the fundamental append-only immutability property required of any secure audit log.

Group 21 suffers from two major vulnerabilities. First, the implementation configures PBKDF2-HMAC-SHA256 with only a single iteration (G21-A2), making offline dictionary and brute-force attacks practical. Using a standard Kali Linux password wordlist and our provided `crack.py` script, we successfully recovered valid authentication tokens from encrypted logs. This is a critical confidentiality failure that also enables subsequent integrity compromise. Second, the implementation contains a race-condition integrity vulnerability (G21-A1) in which concurrent `logappend` processes can corrupt the log state through a Time-of-Check to Time-of-Use (TOCTOU) window, permanently denying service to all subsequent legitimate operations.

These findings represent severe design errors in critical security mechanisms across both implementations.